

Resolución Práctica 7: Clase 04

Grupo 4

Integrantes:

- Avila, Aldana, legajo 1193721, alavila@uade.edu.ar
- Caivano, Alexander Francisco, legajo 1209389, acaivano@uade.edu.ar
- Casareski, Juan Ignacio, legajo 1176109, jcasareski@uade.edu.ar
- Segura, Juan Ignacio, legajo 1242159, jsegura@uade.edu.ar

Parte A: Selección del alcance del grafo

El proceso modelado es "Alta de proveedor", el mismo que se viene trabajando en el TPO desde la definición documental en MongoDB (Práctica 5). Para esta práctica se enriquece el flujo con los caminos alternativos y rechazos que la consigna sugiere, y se agrega un segundo proceso corto ("Baja de proveedor") para mostrar consultas que cruzan procesos.

1. ¿Qué proceso operativo están modelando?
Alta de proveedor de un tenant (ACME S.A.). Registra un proveedor nuevo, valida su documentación, lo aprueba por Compras y, si el monto supera el umbral, lo deriva a Legal antes de notificar el alta.
2. ¿Qué pasos tiene el flujo?
Inicio, formulario inicial, validación documental, revisión de Compras, decisión legal, revisión Legal, notificación y dos finales (aprobado y rechazado).
3. ¿Qué decisiones existen dentro del flujo?
La validación documental (documentación completa o incompleta) y la decisión legal (monto estimado mayor a 1.000.000 o no).
4. ¿Qué roles participan?
Solicitante (carga el formulario), Compras (revisa y aprueba o rechaza) y Legal (revisa los casos de monto alto).
5. ¿Qué caminos alternativos pueden aparecer?
Si la documentación está incompleta, el flujo vuelve al formulario (reproceso). Si el monto no supera el umbral, se saltea la revisión legal y se va directo a notificación.
6. ¿Qué errores, rechazos o derivaciones existen?
Compras puede rechazar y terminar el proceso. Legal puede rechazar y terminar el proceso. La documentación incompleta deriva de vuelta al solicitante.
7. ¿Qué parte del proceso se beneficia de ser representada como grafo?
La topología de transiciones entre pasos. Validar que el flujo no tenga pasos sin salida, detectar ciclos de reproceso, enumerar todos los caminos posibles del inicio al fin y cruzar roles o integraciones entre procesos son recorridos de varios saltos, naturales en un grafo y costosos en un modelo documental.

Parte B: Identificación de nodos

Label	Descripción	Propiedades obligatorias	Propiedades opcionales	Justificación
Tenant	Empresa cliente dueña de los procesos	tenant_id, name	status	Raíz de la propiedad multi-tenant. Todo proceso cuelga de un tenant.

Label	Descripción	Propiedades obligatorias	Propiedades opcionales	Justificación
Process	Proceso BPM configurado y versionado	process_id, name, version	status	Agrupar los pasos de un flujo. La versión permite convivencia de definiciones.
Step	Nodo del flujo (actividad, evento o decisión)	step_id, type, name	order	Es el nodo central del grafo. El campo type (start, form, task, decision, notification, end) define el comportamiento.
Role	Rol que interviene en un paso	role_id, name	n/a	Permite consultar quién participa y reutilizar el rol entre procesos.
Rule	Regla o condición que evalúa una decisión	rule_id, expression	description	Externaliza la lógica de las decisiones para reutilizarla y auditarla.
Integration	Sistema externo invocado por un paso	integration_id, type, name	n/a	Modela dependencias externas compartidas entre varios procesos.

El campo type del Step es la propiedad que más se consulta: separa pasos automáticos de tareas humanas y de decisiones, y marca el inicio y el fin para las consultas de caminos.

Parte C: Identificación de relaciones

Relación	Nodo origen	Nodo destino	Dirección elegida	Propiedades	Justificación
OWNS	Tenant	Process	Tenant hacia Process	created_at	Un tenant posee sus procesos. Aísla los datos por empresa.
HAS_STEP	Process	Step	Process hacia Step	order	El proceso contiene sus pasos. El order da la secuencia base de lectura.
NEXT	Step	Step	Paso origen hacia paso destino	condition, priority	Transición posible entre pasos. Es la relación central: sobre ella corren las consultas de caminos y ciclos. La condition documenta cuándo se toma esa salida.
ASSIGNED_TO	Step	Role	Step hacia Role	required, sla_hours	Asigna una tarea humana a un rol. Permite ver carga por rol y SLA.

Relación	Nodo origen	Nodo destino	Dirección elegida	Propiedades	Justificación
EVALUATES	Step	Rule	Step hacia Rule	on_result	Un paso de decisión evalúa una regla. Separa la condición del paso que la usa.
CALLS	Step	Integration	Step hacia Integration	method, endpoint_name	Un paso invoca un sistema externo. Permite rastrear dependencias compartidas.

La dirección importa porque define el sentido de lectura del flujo. NEXT va siempre del paso anterior al siguiente, así una consulta de caminos avanza en el sentido de ejecución. Las relaciones a Role, Rule e Integration salen del Step porque el paso es quien depende de ellos, no al revés.

Parte D: Diagrama conceptual del grafo

Esquema textual del proceso “Alta de proveedor”. Las flechas con condición indican salidas alternativas de un paso de decisión.

```
(:Tenant {tenant_id:"empresa_acme"})
-[:OWNS]->
(:Process {process_id:"alta_proveedor", version:1})
-[:HAS_STEP]-> (todos los Step de abajo)

(:Step start) -[:NEXT]-> (:Step formulario)
(:Step formulario) -[:NEXT]-> (:Step validacion_docs)
(:Step validacion_docs) -[:NEXT {condition:"docs_completa"}]-> (:Step revision_compras)
(:Step validacion_docs) -[:NEXT {condition:"docs_incompleta"}]-> (:Step formulario)
(:Step revision_compras) -[:NEXT {condition:"aprobado"}]-> (:Step decision_legal)
(:Step revision_compras) -[:NEXT {condition:"rechazado"}]-> (:Step end_rechazado)
(:Step decision_legal) -[:NEXT {condition:"monto>1000000"}]-> (:Step revision_legal)
(:Step decision_legal) -[:NEXT {condition:"monto≤1000000"}]-> (:Step notificacion)
(:Step revision_legal) -[:NEXT {condition:"aprobado"}]-> (:Step notificacion)
(:Step revision_legal) -[:NEXT {condition:"rechazado"}]-> (:Step end_rechazado)
(:Step notificacion) -[:NEXT]-> (:Step end_ok)

(:Step formulario) -[:ASSIGNED_TO]-> (:Role solicitante)
(:Step revision_compras) -[:ASSIGNED_TO {sla_hours:48}]-> (:Role compras)
(:Step revision_legal) -[:ASSIGNED_TO {sla_hours:72}]-> (:Role legal)
(:Step validacion_docs) -[:EVALUATES]-> (:Rule rule_docs)
(:Step decision_legal) -[:EVALUATES]-> (:Rule rule_monto)
(:Step validacion_docs) -[:CALLS {endpoint_name:"validar_cuit"}]-> (:Integration api_afip)
(:Step notificacion) -[:CALLS {endpoint_name:"enviar_mail"}]-> (:Integration smtp_email)
```

El mismo grafo cargado en Neo4j, con todos los pasos y sus relaciones:

Vista de solo las transiciones NEXT, que corresponde al flujo BPM:

El diagrama responde las preguntas de la consigna:

1. ¿Dónde empieza el proceso?
En el Step con type “start” (paso “start”).
2. ¿Dónde termina?
En los Step con type “end”: end_ok (aprobado) y end_rechazado.
3. ¿Qué pasos son obligatorios?
Los que están en todos los caminos del inicio al fin: formulario, validacion_docs y revision_compras.



Figura 1: Grafo completo del proceso alta_proveedor en Neo4j

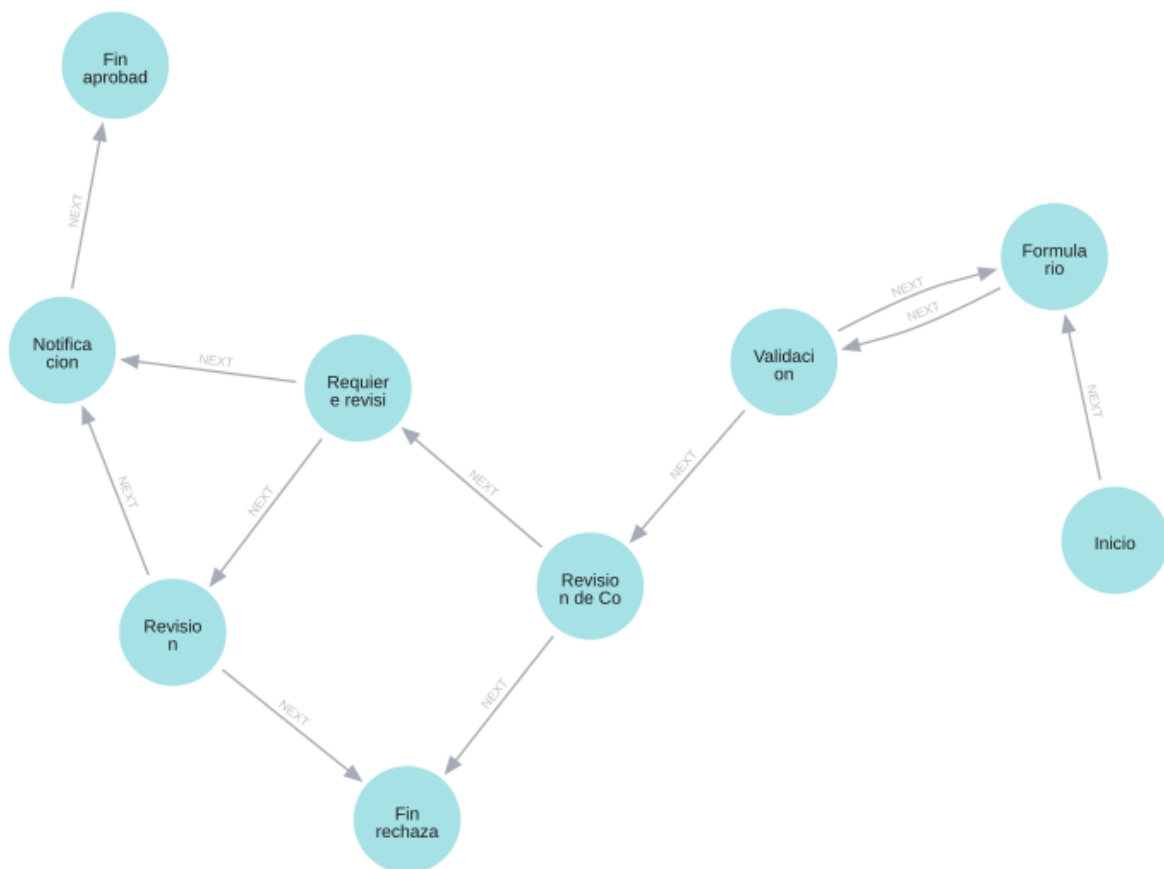


Figura 2: Transiciones NEXT del proceso

4. ¿Qué pasos dependen de condiciones?
Las salidas de validacion_docs y de decision_legal, y los rechazos de revision_compras y revision_legal.
La condición vive en la propiedad condition de la relación NEXT.
5. ¿Qué roles intervienen?
solicitante, compras y legal, vía ASSIGNED_TO.
6. ¿Dónde puede haber caminos alternativos?
En validacion_docs (vuelve al formulario si faltan documentos) y en decision_legal (saltea la revisión legal si el monto no supera el umbral).

Parte E: Consultas Cypher obligatorias

Las consultas se ejecutaron sobre el grafo cargado en Neo4j 5. Se incluye el resultado real de cada una.

N.º	Pregunta que responde	Por qué es útil para FlowOps
1	¿Qué pasos tiene un proceso, en orden?	Reconstruye la definición del flujo para visualizarla o auditarla.
2	¿Cuáles son las transiciones y bajo qué condición se toman?	Documenta el ruteo entre pasos sin recorrer la app.
3	¿Qué pasos requieren intervención humana y de qué rol?	Arma la lista de trabajo por rol y revisa los SLA.
4	¿Hay pasos sin salida (mal definidos o finales)?	Valida la definición antes de publicarla.
5	¿Cuáles son todos los caminos del inicio al fin?	Detecta caminos faltantes o ramas no contempladas.

Consulta 1, pasos del proceso en orden:

```
MATCH (:Process {process_id: "alta_proveedor"})-[:HAS_STEP]->(s:Step)
RETURN s.step_id, s.name, s.type
ORDER BY s.order;
```

Resultado: los nueve pasos del proceso, desde start (type start) hasta end_ok y end_rechazado (type end), pasando por formulario, validacion_docs, revision_compras, decision_legal, revision_legal y notificacion.

Consulta 2, transiciones entre pasos con su condición:

```
MATCH (a:Step)-[:NEXT]->(b:Step)
WHERE a.process_id = "alta_proveedor"
RETURN a.name AS desde, b.name AS hacia, r.condition AS condicion
ORDER BY a.order;
```

Resultado: 11 transiciones. Las normales (start a formulario, notificacion a fin) sin condición, y las de decisión con su condition (docs_completa, docs_incompleta, monto>1000000, rechazado, etc.).

Consulta 3, pasos con intervención humana:

```
MATCH (s:Step {type: "task"})-[:ASSIGNED_TO]->(r:Role)
RETURN s.name AS paso, r.name AS rol;
```

Resultado: Revisión de Compras (rol compras), Revisión Legal (rol legal) y Revisión baja Compras (rol compras, del segundo proceso).

Consulta 4, pasos sin salida:

```
MATCH (s:Step)
WHERE NOT (s)-[:NEXT]->(:Step)
RETURN s.step_id, s.name;
```

Resultado: end_ok, end_rechazado y b_end. Todos son finales legítimos (type end), así que la definición es correcta. Si apareciera un paso que no es final, sería un error de modelado.

Consulta 5, caminos del inicio al fin:

```
MATCH path = (st:Step {process_id: "alta_proveedor", type: "start"})-[:NEXT*]->(en:Step {type: "end"})
RETURN [n IN nodes(path) | n.step_id] AS camino, length(path) AS saltos
ORDER BY saltos;
```

Resultado: cuatro caminos posibles, de 4 a 7 saltos. El más corto termina en rechazo de Compras. El más largo pasa por revisión legal y notificación hasta end_ok. Neo4j no repite una misma relación dentro de un camino, así que el ciclo de reproceso no genera caminos infinitos.

Parte F: Consultas que justifican el uso de grafos

Consulta compleja	Por qué un grafo ayuda	Riesgo si se modela solo como documento
Integraciones externas compartidas por varios procesos	El recorrido Process a Step a Integration cruza procesos en una sola consulta agrupando por integración.	Habría que abrir cada documento de proceso, recorrer sus pasos en la app y armar el cruce a mano. No hay join nativo.
Roles que intervienen en más de un proceso	Mismo recorrido contra Role. La relación es el dato, no requiere tabla puente.	Recorrer todos los documentos y deduplicar roles en código. Cambia según cuántos procesos haya.
Ciclos involuntarios en el flujo	Un patrón de camino que vuelve al mismo nodo detecta el ciclo directo.	En documento hay que reconstruir el grafo en memoria y correr un algoritmo de detección de ciclos por afuera.
Pasos inalcanzables desde el inicio	La negación de un camino de varios saltos lo resuelve en una consulta.	Implica un recorrido transitivo manual sobre el array de edges del JSON.

Consulta F1, integraciones compartidas entre procesos:

```
MATCH (p:Process)-[:HAS_STEP]->(s:Step)-[:CALLS]->(i:Integration)
WITH i, collect(DISTINCT p.process_id) AS procesos
WHERE size(procesos) > 1
RETURN i.name AS integracion, procesos;
```

Resultado: API AFIP, usada por alta_proveedor y baja_proveedor. Sirve para medir el impacto de una caída o un cambio de contrato de esa integración.

Consulta F2, roles que intervienen en más de un proceso:

```
MATCH (p:Process)-[:HAS_STEP]->(s:Step)-[:ASSIGNED_TO]->(r:Role)
WITH r, collect(DISTINCT p.process_id) AS procesos
WHERE size(procesos) > 1
RETURN r.name AS rol, procesos;
```

Resultado: el rol compras participa en alta_proveedor y baja_proveedor. Útil para dimensionar la carga de un rol sobre todos sus procesos.

Consulta F3, ciclos de reproceso en el flujo:

```
MATCH path = (s:Step)-[:NEXT*1..6]->(s)
WHERE s.process_id = "alta_proveedor"
RETURN DISTINCT [n IN nodes(path) | n.step_id] AS ciclo;
```

Resultado: el ciclo formulario a validacion_docs a formulario. Es un reproceso buscado (documentación incompleta vuelve al solicitante), pero la misma consulta delataría un ciclo no deseado.

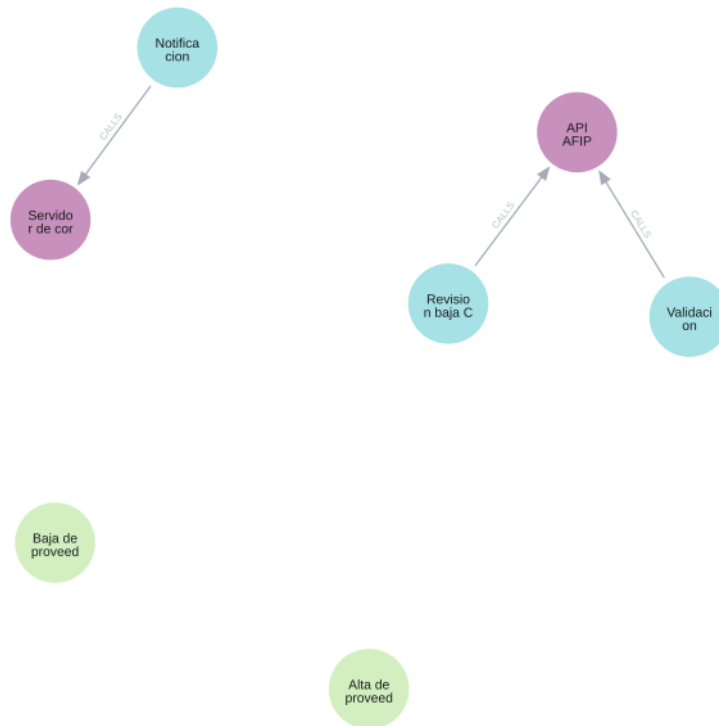


Figura 3: API AFIP compartida entre dos procesos

Consulta F4, pasos inalcanzables desde el inicio:

```

MATCH (st:Step {process_id: "alta_proveedor", type: "start"}),
      (s:Step {process_id: "alta_proveedor"})
WHERE s <> st AND NOT (st)-[:NEXT*]->(s)
RETURN s.step_id AS inalcanzable;

```

Resultado: vacío. Todos los pasos son alcanzables desde el inicio, lo que confirma que el flujo no tiene nodos huérfanos. Un resultado no vacío marcaría un paso que quedó desconectado al editar la definición.

Parte G: Riesgos y límites de usar Neo4j

1. ¿Qué parte de FlowOps NO conviene guardar en Neo4j?
Los datos de carga masiva y los documentos grandes: el historial de eventos, el estado completo de cada instancia en ejecución y los formularios con muchos campos.
2. ¿Guardarían los datos completos de cada instancia en el grafo? ¿Por qué?
No. La instancia es estado mutable de alto volumen y baja conectividad. El grafo aporta valor en la topología de la definición, que es estable y compartida. Mezclar miles de instancias por proceso infla el grafo sin aprovechar las relaciones.
3. ¿Guardarían el historial completo de eventos en el grafo? ¿Por qué?
No. El event log es escritura append-only intensiva, que ya se asignó a Cassandra en la Práctica 6. Neo4j no está pensado para ese patrón de escritura ni para series temporales masivas.
4. ¿Qué datos seguirían estando mejor en MongoDB?
La definición documental versionada del proceso (con formularios y campos dinámicos) y el estado de cada instancia. Son documentos semiestructurados que se consultan por clave, tal como se modeló en la Práctica 5.
5. ¿Qué datos seguirían estando mejor en Cassandra?

El event log y la auditoría: alto volumen de escritura, consulta por tenant_id e instance_id y por rango temporal.

6. ¿Qué problema aparece si intentan usar Neo4j como única base de datos del TPO?

Degrada en los patrones que no son de grafo: escritura masiva de eventos, lectura de estado de baja latencia y documentos grandes. Forzar todo a un grafo agrega complejidad de modelado sin beneficio y deja a Neo4j fuera de su caso fuerte, que son las consultas de conectividad.

Parte H: Integración con el TPO

Subsistema de FlowOps	¿Usaría Neo4j?	Motivo
Definición de procesos	Parcial	La topología (pasos y transiciones) va en el grafo. Los formularios y la versión completa quedan en MongoDB.
Validación de caminos del flujo	Sí	Caso central del grafo: pasos sin salida, ciclos, alcanzabilidad y enumeración de caminos.
Estado actual de instancia	No	Estado mutable de baja latencia. Mejor en MongoDB o Redis.
Eventos de auditoría	No	Escritura append-only masiva. Va en Cassandra.
Tareas humanas pendientes	No	Listas de trabajo por rol que cambian seguido. Mejor en MongoDB o Redis.
Análisis de dependencias entre procesos	Sí	Roles e integraciones compartidos entre procesos se resuelven con recorridos de grafo.
Integraciones externas compartidas	Sí	El cruce paso a integración entre varios procesos es natural en grafo.